From Bayesian Inference to LLMs

Modern C++ Optimizations for Reverse-Mode Automatic Differentiation

STEVE BRONDER



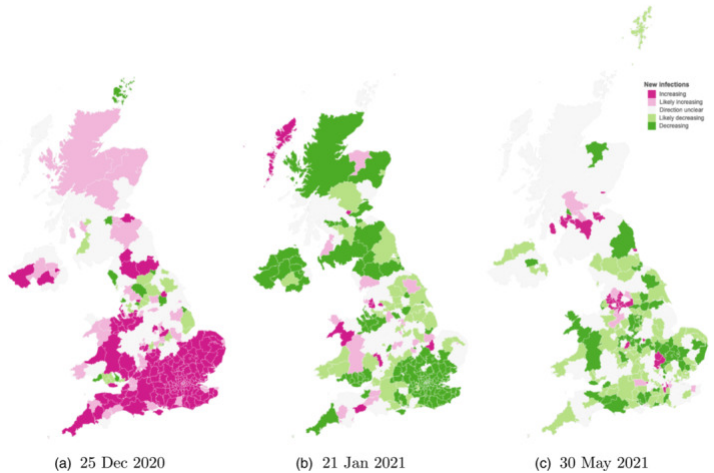


Goal of This talk

- Why you should care about Automatic Differentiation
- How Reverse Mode Automatic Differentiation works
- Performance of high throughput memory intensive programs
- How modern C++ has led to cleaner and more efficient AD

Let v_k be the sequence of intermediate expressions for the input x and output z and let $v_K = z$ and $v_1 = x; v_0 = y$. Let $\overline{v}_k$ be the partial gradient of the kth intermediate step. Then we can apply the chain rule to each intermediate step to get the partial gradient.

Estimating COVID Infection Rates For Policy



(a) 25 Dec 2020 (b) 21 Jan 2021 (c) 30 May 2021

Figure: Probability of epidemic growth by local area

Go to ChatGPT. Autodiff is the mitochondria of machine learning. AD allows for efficient estimation of many statistical models.

Show of Hands

Who has used an LLM?

..

What is Automatic Differentiation?

Automatically compute exact gradients of a function by treating a program as a graph of elementary operations and applying the chain rule on that graph

Reverse-mode automatic differentiation (AD) lets us get derivatives of a program's output with respect to its inputs without doing algebra by hand. Imagine the code as a computational graph of simple ops (add, multiply, sin, etc.). In the forward pass, we execute the code normally, get the output, and save intermediate results. Then we do a backward pass effectively running the chain rule on the graph from the output back to the inputs. This two-pass strategy is what powers deep learning frameworks (it's basically backpropagation), and it efficiently produces gradients even for complex code.

What is a gradient?

Ex: Newton's root finding method

$$x_{n+1} = x_n - \frac{f(x_n)}{f_x{}'(x_n)}$$

find $f(x) = y$ where $y = 0$

- Note that $f_x{}'$ is the gradient of f with respect to x

How do we estimate solutions for equations? A simple method is the newton method.

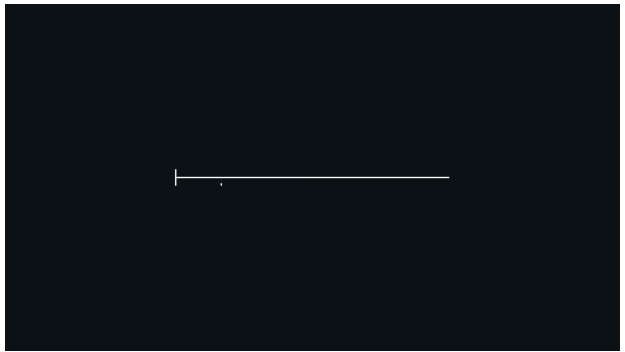
Why are gradients useful? We use gradients to solve

For models we want to estimate parameter values. Change the slide here so that we show how to solve for parameters using Newton's method

What is a gradient?

$$f(x) = x^3 + x^2 + x \tag{1}$$

$$f_x{}'(x) = 3x^2 + 2x + 1 \tag{2}$$



Thank 3blue1brown for manim

The gradient tells us

- The direction the function increases most quickly from x_n
- The rate of increase of the function at x_n

Why use Automatic Differentiation?

- Hamiltonian Monte Carlo:
$$\frac{dp}{dt} = \nabla_\theta \log p(\theta|y)$$
- BFGS:
$$s_k = -H_k \nabla_\theta f(\theta_k)$$
- Stochastic Gradient Descent:
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t; x_t)$$

Hamiltonian Monte Carlo:

- Updates the momentum p using the (negative) gradient of the log posterior—i.e., the deterministic "force" driving the Hamiltonian dynamics.

BFGS:

- Forms the search step s_k as a quasi-Newton descent direction by preconditioning the gradient with the current inverse-Hessian estimate H_k.

Stochastic Gradient Descent:

- Updates parameters by stepping θ a learning-rate η in the direction opposite the stochastic gradient of the loss evaluated on x_t.

Why use Automatic Differentiation?

Choices:

- Write by hand
- finite difference,
- symbolic differentiation
- spectral differentiation
- automatic differentiation

..

Why use Automatic Differentiation?

$$\underbrace{p(\boldsymbol{\theta} \mid \mathrm{y})}_{\textbf{posterior}} \;\propto\; \prod_{i=1}^{N} \Bigg\{ \sum_{\mathrm{z}_i \in \{1,2,3\}^{T_i}} \Bigg[\underbrace{\pi_{z_{i,1}} \prod_{t=2}^{T_i} \Pi_{z_{i,t-1},\, z_{i,t}}}_{\text{3-state HMM prior}} \prod_{t=1}^{T_i} \underbrace{\mathcal{N}\big(y_{i,t} \,\big|\, \eta_{i,t},\, \sigma^2_{z_{i,t}}\big)}_{\text{state-dependent emission}} \Bigg] \Bigg\}$$

where $\eta_{i,t} = \underbrace{\mathrm{x}_{i,t}^{\top}\boldsymbol{\beta}}_{\text{fixed}} + \underbrace{\mathrm{z}_{i,t}^{(G)\top}\mathrm{b}_{g[i]} + \mathrm{z}_{i,t}^{(C)\top}\mathrm{c}_{c[i]} + u_{g[i]}}_{\text{crossed random effects}} + \underbrace{f(t_{i,t})}_{\text{GP}} + \underbrace{\mu_{z_{i,t}} + \mathrm{r}_{z_{i,t}}^{\top}\mathrm{w}_i}_{\text{state-specific offset + slope}}, \qquad z_{i,t} \in \{1,2,3\}.$

$p(\mathrm{f} \mid \boldsymbol{\psi}) = (2\pi)^{-T/2}\, |\mathbf{K}|^{-1/2}\, \exp\Big(-\tfrac{1}{2}\,\mathrm{f}^{\top}\mathbf{K}^{-1}\mathrm{f}\Big),$

$\mathbf{K} = \sigma_f^2\Big(\mathbf{K}_{\mathsf{LP}}(\ell, p, \lambda) \;+\; \rho\,\mathbf{K}_{\mathsf{SE}}(\tilde{\ell})\Big) + \sigma_n^2\mathbf{I}, \quad \big[\mathbf{K}_{\mathsf{LP}}\big]_{tt'} = \exp\left(-\frac{(t-t')^2}{2\ell^2} - \frac{2\sin^2\big(\pi|t-t'|/p\big)}{\lambda^2}\right),$

$\big[\mathbf{K}_{\mathsf{SE}}\big]_{tt'} = \exp\left(-\frac{(t-t')^2}{2\tilde{\ell}^2}\right), \qquad \mathbf{K} = \mathbf{L}_K\mathbf{L}_K^{\top} \;\Rightarrow\; \log|\mathbf{K}| = 2\sum_{j=1}^{T}\log\big((\mathbf{L}_K)_{jj}\big).$

Hierarchical mixed effects (non-centered, LKJ prior):

$\mathrm{b}_g = (\mathbf{I}_{p_G} \otimes \mathrm{diag}(\boldsymbol{\tau}_b)\,\mathbf{L}_R)\,\tilde{\mathrm{b}}_g, \quad \tilde{\mathrm{b}}_g \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathrm{c}_c = \mathrm{diag}(\boldsymbol{\tau}_c)\,\tilde{\mathrm{c}}_c, \quad \tilde{\mathrm{c}}_c \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$

$\mathrm{LKJ}_{p_G}(\eta)$ prior on $\mathbf{R}, \quad \mathbf{L}_R\mathbf{L}_R^{\top} = \mathbf{R}, \quad \boldsymbol{\tau}_b \sim \prod_{j=1}^{p_G} \text{Half-}t_{\nu_b}(0, s_b), \quad \boldsymbol{\tau}_c \sim \prod_{j=1}^{p_C} \text{Half-}t_{\nu_c}(0, s_c),$

$u_g \sim \mathcal{N}(0, \sigma_u^2).$

Move from this slide quickly

Why use Automatic Differentiation?

- I do not wish to write those gradients by hand

.....

Why use Automatic Differentiation?

- I do not wish to write those gradients by hand
- Instead of N finite differences, a forward and backward pass

.....

Why use Automatic Differentiation?

- I do not wish to write those gradients by hand
- Instead of N finite differences, a forward and backward pass
- Accurate to floating point precision

.....

Why use Automatic Differentiation?

- I do not wish to write those gradients by hand
- Instead of N finite differences, a forward and backward pass
- Accurate to floating point precision
- Allows for unknown length while and for loops

.....

Implementation Matters!

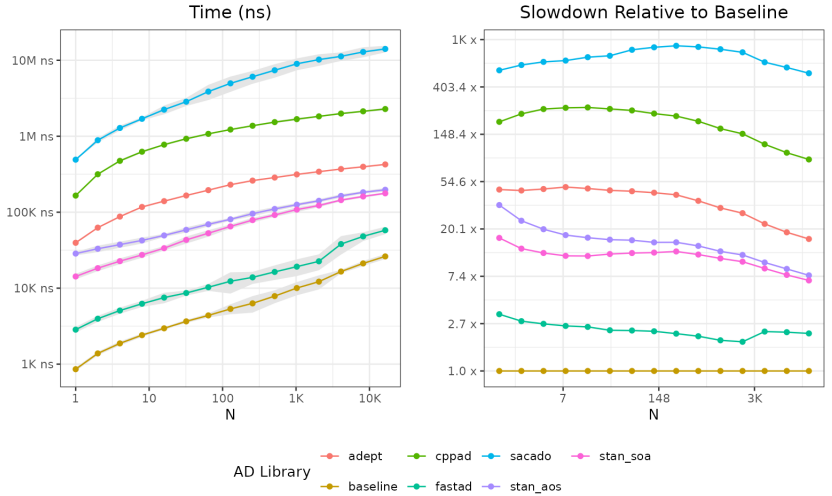


Figure: Log-Log Benchmark for f and f' given $f(y) = N(y|X\theta, \sigma)$

..

What is Automatic Differentiation?

- AD computes gradients of a program by applying the chain rule to its subexpressions.

$$
\begin{aligned}
f(x, y) &= \log(x \cdot y) \\
g(x, y) &= x \cdot y \\
h(u) &= \log(u) \\
f(x, y) &= h\big(g(x, y)\big)
\end{aligned}
$$

- Emphasize: AD is mechanical chain rule over the executed program, not symbolic math or numeric differencing.
- Automatic Differentiation can do higher order partials, but here we just focus on the first order
- Now we rewrite f in terms of h and g

What is Automatic Differentiation?

- AD computes gradients of a program by applying the chain rule to its subexpressions.

$$
\begin{aligned}
f(x,y) &= h\big(g(x,y)\big) \\
g(x,y) &= x \cdot y \qquad\qquad {g_x}'(x,y) = y \;\; {g_y}'(x,y) = x \\
h(u) &= \log(u) \qquad\qquad h'_u(u) = u^{-1}
\end{aligned}
$$

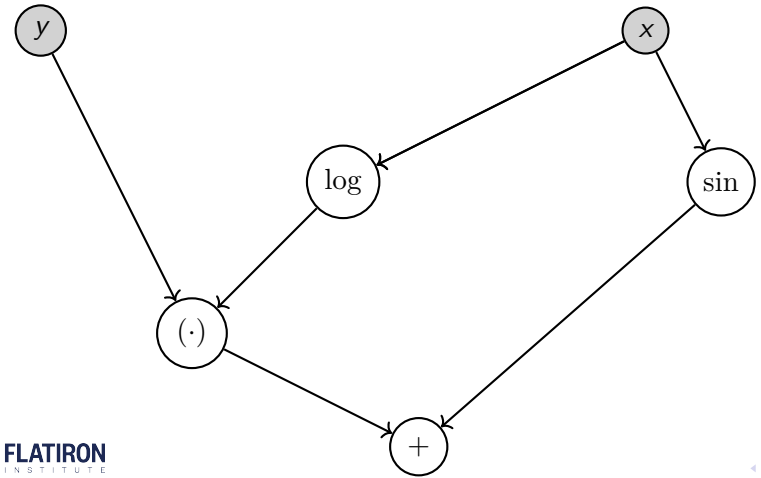
$$
\begin{aligned}
f'_x(x,y) &= h'_g\big(g(x,y)\big) \cdot g'_x(x,y) \\
&= \frac{1}{g(x,y)} \cdot y = \frac{1}{x}, \\
f'_y(x,y) &= h'_g\big(g(x,y)\big) \cdot g'_y(x,y) \\
&= \frac{1}{g(x,y)} \cdot x = \frac{1}{y}.
\end{aligned}
$$

Emphasize: AD is mechanical chain rule over the executed program, not symbolic math or numeric differencing.

Data Type: Expression Graph

Goal: Calculate the full gradient by accumulating partial gradients (adjoints) through the a graph of subexpressions.

$$z = \log(x) \cdot y + \sin(x)$$



- Each node of expression graph is an intermediate calculation we know the gradient of with respect to its inputs
- For each node we compute a forward pass till we get to our end node
- Once we reach our end node we can compute the reverse pass

Expression Graph: Traversal

$$f(x, y) = \log(x)y + \sin(x)$$

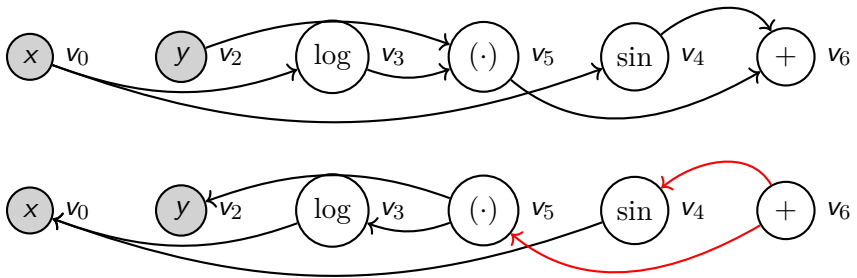


Figure: Topological sort of expression graph

- We can do a Topological sort on the graph and traverse it this way
- This is how we should execute the program linearly on a thread

Expression Graph: Node

For Reverse Mode AD, each node performs two functions.

- Forward Pass:

$$z_{child} = f(z_{parent_1}, z_{parent_2})$$

Adjoint = local gradient

- Each node in the graph holds an adjoint value, which is the partial derivative of the final output with respect to that node's value. It accumulates all contributions of that node to the output's gradient.

Backpropagation (chain rule):

- During the backward pass, each operation uses its adjoint and the operation's local derivative to add to its input nodes' adjoints. This is how gradients flow backward through the graph

Expression Graph: Node

For Reverse Mode AD, each node performs two functions.

- Forward Pass:

$$z_{child} = f(z_{parent_1}, z_{parent_2})$$

- Reverse Pass:
  Given z_{child}'s adjoint (partial gradient) $\bar{z}_{child}$
  Calculate the local adjoint update for z_{parent_1} and z_{parent_2}.

$$chain(\bar{z}_{child}, z_{parent_1}, z_{parent_2}) = \begin{cases} \bar{z}_{parent_1} \mathrel{+}= \bar{z}_{child} \cdot z'_{parent_1} \\ \bar{z}_{parent_2} \mathrel{+}= \bar{z}_{child} \cdot z'_{parent_2} \end{cases}$$

Adjoint = local gradient

- Each node in the graph holds an adjoint value, which is the partial derivative of the final output with respect to that node's value. It accumulates all contributions of that node to the output's gradient.

Backpropagation (chain rule):

- During the backward pass, each operation uses its adjoint and the operation's local derivative to add to its input nodes' adjoints. This is how gradients flow backward through the graph

Pseudocode for AutoDiff

```
// compute and store node.value
struct node {
  double val;
  double adj{0};
  virtual void chain() {}
};
node x = 2.0;
node y = 3.0;
// Forward pass calculated here
auto expr_graph = log(x) * y + sin(x);
// Calculate reverse pass
e_graph.end().adj = 1.0;
for (auto& expr : expr_graph | std::views::reverse) {
    expr->chain();
}
assert(y.adj == 1.2103);
assert(x.adj == 0.2516);
```

What Do AD Libraries Care About?

- Flexibility:
  Debugging, exceptions, conditional loops, matrix assignment
- : Efficiency:
  Efficiently using a single CPU/GPU
- Scaling
  Efficiently using clusters with multi-gpu/cpu nodes

C++ gives us the tools to meaningfully make these tradeoffs decisions.

From Rachel: C++ is great for tools for writing high levels abstractions over lower level code

How do we keep track of our reverse pass?

Operator Overloading

- Nodes in the expression graph are objects which store a forward and reverse pass function

Good: Easier to implement, more flexible

Bad: Less optimization opportunities

Source code transformation

- Unroll all forward passes and reverse passes into one function

Good: Fast

Bad: Hard to implement, very restrictive

flexibility, performance, and developer time

OO is runtime and we need allocators and to think about cache locality

- SCT is compile time and we need to think about expression templates and template metaprogramming
- Known expression graph size at compile time? (Static)
- Reassignment of variables (Dynamic easy, Static hard)
- How much time do I have? (Dynamic)
- C++ is great for having dynamic or static

Operator Overloading Approach

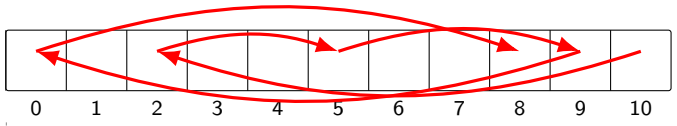
The operator overloading approach usually involves:

- Pair scalar type to hold the value and adjoint
- Runtime "tape" for tracking expressions
- Arena allocator for storing AD nodes

The main enemy of dynamic tapes is cache locality

Issues

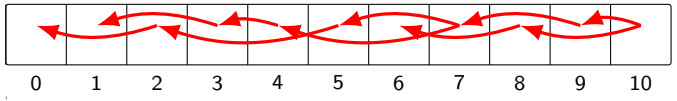
Noncontiguous Tape



This is what we want to have, but more often than not we will still end up with jumps

Issues

Contiguous Tape



...

Operator Overloading: Monotonic Buffer

```
struct var_impl;
struct Tape {
std::monotonic_buffer_resource mbr{(1 << 16)};
std::polymorphic_allocator<std::byte> pa{&mbr};
std::vector<var_impl*> tape;
void clear() {
  tape.clear();
  mbr.release();
}
};
static Tape g_tape{};
```

We want a big block of memory that we reuse over and over

Same calculations are done over and over with new values

Operator Overloading: Basic Type

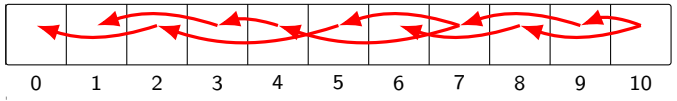
```
struct var_impl {
  double val;
  double adj;
  virtual void chain() {};
}; // 24 bytes
struct var {
  var_impl* vi_;
}; // 8 bytes
```

var_impl is our base type all expression nodes inherit from

Why vptr? Mainly optimization

Issues

Contiguous Tape



...

Operator Overloading: Expression Node

```
struct mul_vv : public var_impl {
  var op1_;
  var op2_;
  void chain() {
    op1_.adj += this->adjoint_ * op2_.val;
    op2_.adj += this->adjoint_ * op1_.val;
  }
}; // 40 bytes
```

..

Operator Overloading: Monotonic Buffer

```
template <typename T, typename Types>
auto new_node(Types&&... args) {
  auto* ret = g_tape.pa.template new_object<T>(args...);
  g_tape.tape.push_back(ret);
  return var{ret};
}
```

..

Operator Overloading: Expression Node

```
var operator*(var op1, var op2) {
  return new_node<mul_vv>(op1.val() * op2.val(),
    op1, op2);
}
```

..

Operator Overloading: Reverse Pass

```
void grad(var& v) {
  v.adj = 1.0;
  for (auto&& expr : tape | std::views::reverse) {
    expr->chain();
  }
}
```

..

Operator Overloading: Example

```
void compute_grads(var x, var y) {
  var z = -10;
  while (z.val() < 20) {
    z += log(x * y);
  }
  grad(z);
}
```

..

Operator Overloading: Example

```
// Later in program
for (int i = 0; i < 1e10; ++i) {
  var x = compute_x(...);
  var y = compute_y(...);
  compute_grads(x, y);
  do_something_with_grads(x.adj, y.adj);
  g_tape.clear();
}
```

..

So Many Operator Classes

```
struct mul_vv;
struct mul_vd;
struct mul_dv;
struct add_vv;
struct add_dv;
struct add_vd;
struct subtract_vv;
struct subtract_dv;
struct subtract_vd;
struct divide_vv;
struct divide_dv;
struct divide_vd;
```

..

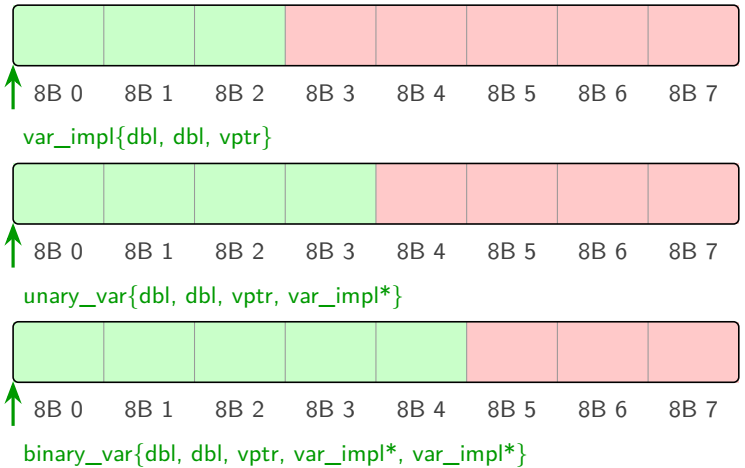
Reduce Boilerplate

```cpp
template <Functor F>
struct callback_var_impl : public var_impl {
  F rev_functor_;
  template <std::floating_point S>
  explicit callback_var_impl(
    S&& value, F&& rev_functor)
    : var_impl(value),
      rev_functor_(rev_functor) {}
  void chain() final { rev_functor_(*this); }
}; // 24 + 8B * N
```

Reduce Boilerplate

```
template <typename T1, typename T2>
requires any_var<T1, T2>
inline auto operator*(T1 op1, T2 op2) {
  return new_node<callback_var_impl>(
    value(op1) * value(op2),
    [op1, op2](auto&& ret) mutable {
      if constexpr (is_var_v<T1>) {
        adjoint(op1) += adjoint(ret) * value(op2);
      }
      if constexpr (is_var_v<T2>) {
        adjoint(op2) += adjoint(ret) * value(op1);
      }
    });
}
```

Poor Cache Use



Tracking everything dynamiclaly is very expensive

Source Code Transform Example:

```
double z = log(x * y);
```

Break it down

```
double v0 = x;
double v1 = y;
double v2 = x * y;
double v3 = log(v2)
double bar_v3 = 1;
double bar_v2 = bar_v3 * 1/v2;
double bar_v1 = bar_v2 * v0;
double bar_v0 = bar_v2 * v1;
```

Fortran AD transpilers first famous for this

We can leverage C++'s type system to do this at compile time

Source Code Transform Base Type:

```
struct var {
    double values_;
    double adjoints_;
    var(double x) : values_(x), adjoints_(0) {}
};
```

..

Source Code Transform Expression Node

```
template <typename F, typename... Exprs>
struct expr {
   var ret_;
   std::tuple<deduce_ownership_t<Exprs>...> exprs_;
   std::decay_t<F> f_;
   template <typename FF, typename... Args>
   expr(double x, FF&& f, Args&&... args) :
     ret_(x), f_(std::forward<F>(f)),
     exprs_(std::forward<Args>(args)...) {}
};
```

..

Source Code Transform Operator

```cpp
template <typename T1, typename T2>
requires any_var_or_expr<T1, T2>
inline auto operator*(T1&& lhs, T2&& rhs) {
  return make_expr(value(lhs) * value(rhs),
  [](auto&& ret, auto&& lhs, auto&& rhs) {
    if constexpr (is_var_v<T1>) {
      adjoint(lhs) += adjoint(ret) * value(rhs);
    }
    if constexpr (is_var_v<T2>) {
      adjoint(rhs) += adjoint(ret) * value(lhs);
    }
  }, std::forward<T1>(lhs), std::forward<T2>(rhs));
}
```

Source Code Transform Expression Graph

```
auto z = x * log(y) + log(x * y) * y;
// Polish Notation
// + (* x (log y)) (* (log (* x y)) y)
expr<Lambda<Plus>,
  expr<Lambda<Mult>, var, expr<Lambda<Log>, var>>,
  expr<Lambda<Mult>,
   expr<Lambda<Log>, expr<Lambda<Mult>, var, var>>,
   var>>
```

Thinking about this in polish notation lets us think about the expression templates as a stack we go through

Source Code Transform Gradient

```cpp
template <typename Expr>
inline void grad(Expr&& z) {
  adjoint(z) = 1.0;
  // Tuple nodes to be iterated in breadth-first order
  // <expr<Plus, expr<Mult>, expr<Mult>>,
  //  expr<Mult, var, expr<Log>>,
  //  expr<Mult, expr<Log>, expr<Mult>> ...
  auto nodes = collect_bfs(z);
  eval_breadthwise(nodes);
}
```

Example Godbolt

mono buff: 811

sct: 47

Scalar Comparison

Table: $f(x, y) = x\log(y) + \log(xy)y$;

Method	CPU Time	Slowdown
Dynamic	47.8ns	9x
Source Code Transform	6.3ns	.21x
Baseline	5.23ns	

..

Source Code Transform: Downsides

Code like the following very hard / impossible in source code transform

```
while(error < tolerance) {
  var z += log(x * y);
}
```

Since the SCT needs to know how all of the nodes at compile time

Operator Overloading Approach: Matrices

```
Matrix<var> B(M, M);
Matrix<var> X(M, M);
Matrix<var> Z = X * B.transpose();
```

For most models a large part of the time will be spent on matrix operations.

Models like the UK covid forecasting model have large autodiff matrices

Operator Overloading Approach: Matrices

```
template <typename MatrixType>
struct arena_matrix :
  public Eigen::Map<MatrixType> {
  using Base = Eigen::Map<MatrixType>
  template <typename T>
  arena_matrix(T&& mat) :
   Base(copy_to_arena(mat.data(), mat.size()),
     mat.rows(), mat.cols()) {}
};
```

..

Operator Overloading Approach: Matrices

```
// Array of Structs
struct Matrix<var> {
  var* data_;
};
// Struct of Arrays
struct var_impl<Matrix<double>> {
  arena_matrix<double> value_;
  arena_matrix<double> adjoint_;
  virtual void chain() {}
};
```

..

Operator Overloading Approach: Matrices

```
// Array of Structs
struct Matrix<var> {
  var* data_;
};
```

- Array of Structs:
  - Simple, most algorithms Just Work™
  - Adds a lot to expression graph
  - turns off SIMD

..

Operator Overloading Approach: Matrices

```
// Struct of Arrays
struct var_impl<Matrix<double>> {
  arena_matrix<double> value_;
  arena_matrix<double> adjoint_;
  virtual void chain() {}
};
```

- Struct of Arrays:
  - Hard, everything written out manually
  - Collapses matrix expressions in tree
  - SIMD can be used on values and adjoints

Matrix Multiplication Example

```
template <typename T1, typename T2>
requires any_var_matrix<T1, T2>
inline auto operator*(T1&& op1, T2&& op2) {
  return lambda_var(value(op1) * value(op2),
    [op1, op2](auto&& ret) mutable {
      if constexpr (is_var_matrix_v<T1>) {
        adjoint(op1) += adjoint(ret) * value(op2).transpose();
      }
      if constexpr (is_var_matrix_v<T2>) {
        adjoint(op2) += value(op1) * adjoint(ret);
      }
    });
}
```

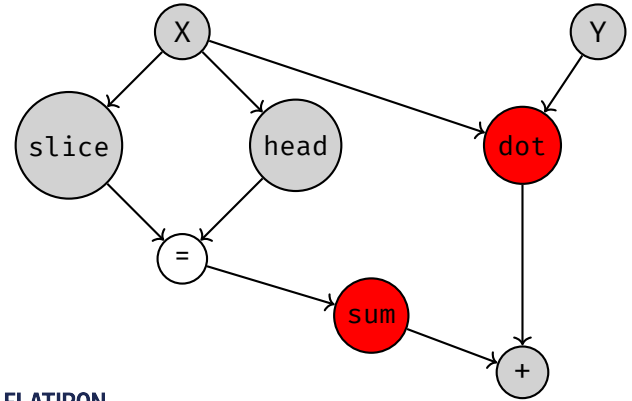
..

Subset Assignment

```
var<Vector<double>> y{{0, 1, 2, 3}};
var<Vector<double>> x{{0, 1, 2, 3}};
var prod = y.dot(x);
x.slice(1, 3) = x.head(3);
auto z = prod + sum(x);
```

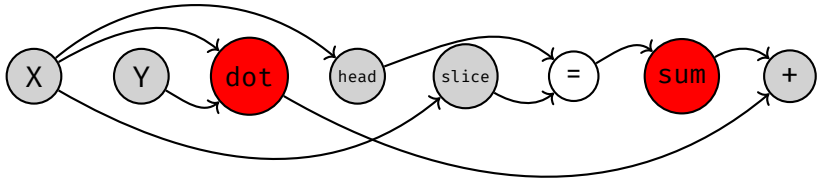
Subset Assignment

```
var<Vector<double>> y{{0, 1, 2, 3}};
var<Vector<double>> x{{0, 1, 2, 3}};
var prod = y.dot(x);
x.slice(1, 3) = x.head(3);
auto z = prod + sum(x);
```



Subset Assignment

```
var<Vector<double>> y{{0, 1, 2, 3}};
var<Vector<double>> x{{0, 1, 2, 3}};
var prod = y.dot(x);
x.slice(1, 3) = x.head(3);
auto z = prod + sum(x);
```



Subset Assignment Becomes Very Hard

```
var<Vector<double>> x{{0, 1, 2, 3}};
x.slice(1, 3) = x.head(3);
```

Iter	X
init	{0, 1, 2, 3}
0	{0, 0, 2, 3}
1	{0, 0, 0, 3}
2	{0, 0, 0, 3}

Aliasing is a bad thing that can happen in our subset assignment

At the assign op level, we don't know if the lhs and rhs are the same object

Subset Assignment Becomes Very Hard

```
tempalte <typename T>
struct var {
template <typename S>
require AssignableExpression<T, S>
inline var<T>& operator=(const var<S>& other) {
  arena_matrix<T> prev_val(vi_->val_.rows(), vi_->val_.cols());
  prev_val.deep_copy(vi_->val_);
  vi_->val_.deep_copy(other.val());
  g_tape.callback(
  [this_vi = this->vi_, other_vi = other.vi_, prev_val]() mutable {
    this_vi->val_.deep_copy(prev_val);
    prev_val.deep_copy(this_vi->adj_);
    this_vi->adj_.setZero();
    other_vi->adj_ += prev_val;
  });
  return *this;
}
}
```

Do not worry about the code here. The main thing you need to see is that subset assignment is very expensive and requires a ton of deep copies of nodes in our expression graph.

Source Code Transform: Matrices

```
ad::Var X(4, 4); // Placeholders
ad::Var Y(4, 4);
auto expr = sum(X * Y);
auto [val_size, adj_size] = expr.cache_sizes();
Eigen::VectorXd val_buf(val_size);
Eigen::VectorXd adj_buf(adj_size);
expr.bind(std::make_pair(val_buf, 0), std::make_pair(adj_buf, 0));
```

Just like the scalar source code transform we can do the same thing for matrices.

We can also get rid of the static global tape and use a local cache instead.

Matrix Expression Graph Gradient

```
template <typename Expr>
void grad(Expr&& expr,
  std::pair<MatrixXd, MatrixXd>& X,
  std::pair<MatrixXd, MatrixXd>& Y) {
  expr.reset_adjoints();
  ad::autodiff(expr, ad::VarView(X.first, X.second),
    ad::VarView(Y.first, Y.second));
}
```

..

Placeholder Node

```cpp
template <typename T>
struct Var {
  double* val_;
  double* adj_;
  std::size_t rows_;
  std::size_t cols_;
  static constexpr std::size_t ops = 1;
  Var(std::size_t rows, std::size_t cols)
    : rows_(rows), cols_(cols) {}
  inline auto Forward(VarView<T>& x) {
    val_ = x.val_.data();
    adj_ = x.adj_.data();
    return Map<MatrixXd>(val_, rows_, cols_);
  }
  template <typename Arg>
  inline void Reverse(Arg& child_adj) {
    Map<MatrixXd>(adj_, rows_, cols_) += child_adj;
  }
}
```

..

Matrix Multiplication Expression Node

```
template <Matrix Op1, Matrix Op2>
struct MatMul {
  static constexpr std::size_t ops = 2;
  MatMul(const Op1& op1, const Op2& op2) :
    op1_(op1), op2_(op2) {}
  template <typename... Args>
  inline auto Forward(Args&&... args);
  template <typename Arg>
  inline auto Reverse(Arg&& child_adj);
  std::pair<std::size_t, std::size_t> cache_sizes();
  void bind_cache(double* val_buf, double* adj_buf,
   std::size_t val_pos, std::size_t adj_pos);
  Op1 op1_;
  Op2 op2_;
  double* val_;
  double* adj_;
  std::size_t rows_;
  std::size_t cols_;
};
```

Expression Forward Pass

```
template <typename... Args>
inline auto Forward(Args&&... args) {
  auto op1_v = op1_.Forward(
    slice<0, Op1::ops>(args)...);
  auto op2_v = op2_.Forward(
    slice<Op1::ops, Op2::ops>(args)...);
  return this->val_ = op1_v * op2_v;
}
```

..

Expression Reverse Pass

```
template <Matrix Op1, Matrix Op2>
struct MatMul {
  template <typename Arg>
  inline auto Reverse(Arg&& child_adj) {
    this->adj += child_adj;
    auto left_adj = op1_.transpose() * this->adj;
    op2_.Reverse(left_adj);
    auto right_adj = this->adj * op2_.transpose();
    op1_.Reverse(right_adj);
  }
};
```

..

Expression Cache Size

```
template <Matrix Op1, Matrix Op2>
struct MatMul {
  inline auto cache_bind_size(
    std::pair<size_t, size_t>& child_sizes) {
    auto op1_sizes = op1_.cache_sizes();
    auto op2_sizes = op2_.cache_sizes();
    return std::make_pair(
      op1_sizes.first + op2_sizes.first + this->val_.size(),
      op1_sizes.second + op2_sizes.second + this->adj_.size());
  }
};
```

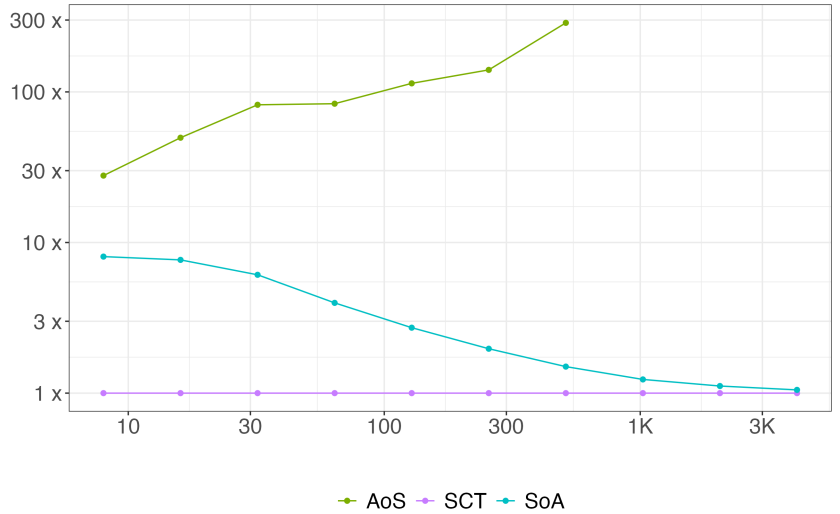
Expression Cache Size

```
template <Matrix Op1, Matrix Op2>
struct MatMul {
  void bind_cache(
    std::pair<double, size_t>& val_buf,
    std::pair<double*, size_t>& adj_buf) {
    this->val_ = val_buf.first + val_buf.second;
    this->adj_ = adj_buf.first + adj_pos.second;
    val_buf.second += this->val_.size();
    adj_buf.second += this->adj_.size();
    op1_.bind_cache(val_buf, adj_buf);
    op2_.bind_cache(val_buf, adj_buf);
  }
};
```

Matrix Expression Graph Creation

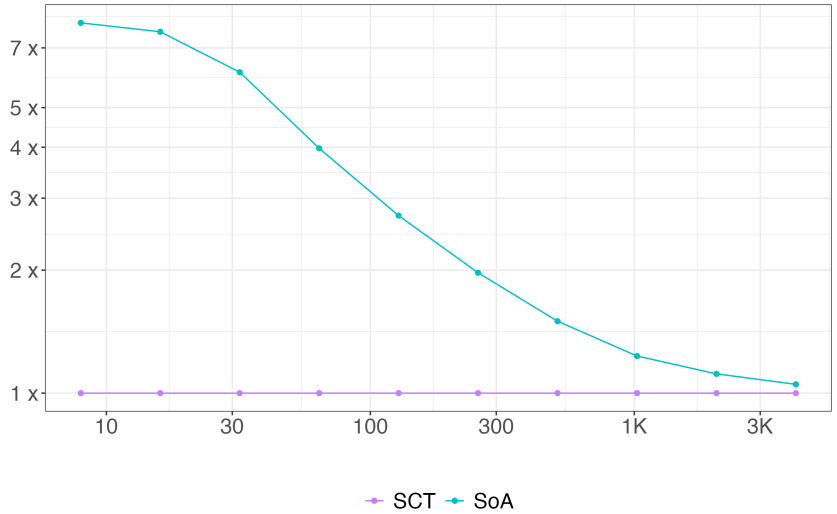
```
ad::Var X(4, 4); // Placeholders
ad::Var Y(4, 4);
auto expr = sum(X * Y);
auto [val_size, adj_size] = expr.cache_sizes();
Eigen::VectorXd val_buf(val_size);
Eigen::VectorXd adj_buf(adj_size);
expr.bind(std::make_pair(val_buf, 0), std::make_pair(adj_buf, 0));
ad::autodiff(expr,
  ad::VarView(X_v.first, X_v.second),
  ad::VarView(Y_v.first, Y_v.second));
```

Matrix Multiplication Benchmark



This is sum(X * Y) where X and Y have NxN elements

Matrix Multiplication Benchmark



..

Speedup Holds for Larger Expression Graphs

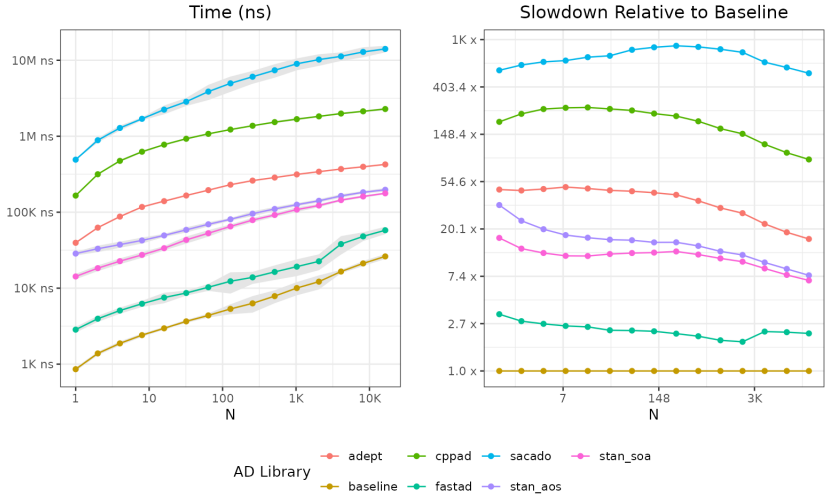


Figure: Log-Log Benchmark for f and f' given $f(y) = N(y|X\theta, \sigma)$

As are expression graph grows the SCT matrix form continues in performance

Thanks!

Repository for benchmarks and slides



The point is that there is still really cool research to do in autodiff

Design decisions lead to no free lunch, but there are many options